Identificação de Atribuições em Linguagem C

Análise do programa `ordena_vetor.c` *(ordenação de vetor, soma e média)*

1. Objetivo

Identificar e contar as atribuições presentes no programa `ordena_vetor.c`, que lê cinco números inteiros, ordena o vetor pelo método da bolha (*bubble sort*) e exibe os valores ordenados, a soma e a média. A análise segue o mesmo método do exemplo de verificação de palíndromo, classificando cada atribuição como **direta** ou **indireta**.

2. Fundamentação teórica

Em linguagem C, **atribuir** é fazer com que um local de memória (variável, parâmetro ou posição de vetor) passe a guardar um valor. Isso pode ocorrer de duas formas:

Atribuição direta	**Atribuição indireta**
Aparece escrita no código com o operador `=`. A variável à esquerda recebe o valor da expressão à direita. A inicialização na declaração (`int total = 0;`) também é atribuição direta. Exemplo: `total = total + v[i];`	Um valor é copiado **sem o sinal** `=`. Ocorre em dois casos: (a) na **chamada de função**, quando cada argumento é copiado para o parâmetro correspondente; (b) no **return**, quando o valor retornado é entregue a quem chamou a função. Exemplo: `ordena(numeros, TAM);`

Em C a passagem de parâmetros é feita **por valor**: a função recebe uma *cópia* do argumento. Quando o argumento é um vetor (como `numeros`) ou um endereço (como `&v[j]`), o que é copiado para o parâmetro é o **endereço**. Por isso, o parâmetro passa a apontar para os mesmos dados do chamador.

3. Critérios de contagem

Para tornar a contagem reproduzível, adotaram-se as regras abaixo, as mesmas do exemplo do palíndromo:

R1	Cada ocorrência de `=` (inclusive na inicialização de variável e nos laços `for`) conta **1 atribuição direta**.
R2	Cada **chamada de função** conta **1 atribuição indireta**, independentemente do número de argumentos (ex.: `scanf("%d", &x)` conta 1).
R3	Cada **return** conta **1 atribuição indireta**.
R4	**Não contam**: declarações sem valor inicial (`int i, j;`), comparações (`<`, `>`, `==`) e diretivas de pré-processador (`#define`).

Exemplo resolvido (linha 44). A linha `resultado = soma(numeros, TAM);` contém **duas** atribuições:

[20] indireta: os argumentos `numeros` e `TAM` são copiados para os parâmetros `v` e `n` da função `soma`.

[19] direta: o valor devolvido por `soma` (via `return total`) é guardado em `resultado` pelo operador `=`.

4. Código-fonte analisado

Legenda: trechos em **verde** são atribuições diretas; trechos em **laranja** são atribuições indiretas. O número entre colchetes é o identificador usado na Seção 5.

```c
#include <stdio.h>

#define TAM 5

void troca(int *a, int *b) {
    int aux = *a[1];
    *a = *b[2];
    *b = aux[3];
}

void ordena(int v[], int n) {
    int i, j;

    for (i = 0[4]; i < n - 1; i = i + 1[5]) {
        for (j = 0[6]; j < n - 1 - i; j = j + 1[7]) {
            if (v[j] > v[j + 1]) {
                troca(&v[j], &v[j + 1])[8];
            }
        }
    }
}

int soma(int v[], int n) {
    int total = 0[9];
    int i;

    for (i = 0[10]; i < n; i = i + 1[11]) {
        total = total + v[i][12];
    }
    return total[13];
}

int main(void) {
    int numeros[TAM];
    int i, resultado;
    float media;

    printf("Digite %d numeros inteiros:\n", TAM)[14];
    for (i = 0[15]; i < TAM; i = i + 1[16]) {
        scanf("%d", &numeros[i])[17];
    }

    ordena(numeros, TAM)[18];
    resultado =[19] soma(numeros, TAM)[20];
    media = (float) resultado / TAM[21];

    printf("Vetor ordenado:")[22];
    for (i = 0[23]; i < TAM; i = i + 1[24]) {
        printf(" %d", numeros[i])[25];
    }
    printf("\nSoma: %d\nMedia: %.2f\n", resultado, media)[26];

    return 0[27];
}
```

Figura 1 – Código `ordena_vetor.c` *com as atribuições marcadas.*

5. Análise detalhada por função

A tabela organiza as 27 atribuições por função, na ordem em que aparecem no código. A coluna "Explicação" indica **quem recebe** e **o que recebe**.

#	Linha	Tipo	Trecho	Explicação
Função troca (linhas 5 a 9) – troca o conteúdo de duas posições				
1	6	**Direta**	`aux = *a`	aux recebe o valor apontado por a.
2	7	**Direta**	`*a = *b`	A posição apontada por a recebe o valor apontado por b.
3	8	**Direta**	`*b = aux`	A posição apontada por b recebe o valor guardado em aux.
Função ordena (linhas 11 a 21) – ordenação pelo método da bolha				
4	14	**Direta**	`i = 0`	i recebe 0 (laço externo).
5	14	**Direta**	`i = i + 1`	i recebe o próprio valor mais 1.
6	15	**Direta**	`j = 0`	j recebe 0 (laço interno).
7	15	**Direta**	`j = j + 1`	j recebe o próprio valor mais 1.
8	17	**Indireta**	`troca(&v[j], &v[j + 1])`	Os endereços de v[j] e v[j+1] são copiados para os parâmetros a e b.
Função soma (linhas 23 a 31) – soma os elementos do vetor				
9	24	**Direta**	`total = 0`	total recebe 0.
10	27	**Direta**	`i = 0`	i recebe 0.
11	27	**Direta**	`i = i + 1`	i recebe o próprio valor mais 1.
12	28	**Direta**	`total = total + v[i]`	total acumula o elemento atual do vetor.
13	30	**Indireta**	`return total`	O valor da soma é devolvido a quem chamou e chega a resultado na linha 44.
Função main (linhas 33 a 54) – leitura, chamadas e exibição				
14	38	**Indireta**	`printf("Digite…", TAM)`	O texto e o valor de TAM são copiados para os parâmetros de printf.
15	39	**Direta**	`i = 0`	i recebe 0.
16	39	**Direta**	`i = i + 1`	i recebe o próprio valor mais 1.
17	40	**Indireta**	`scanf("%d", &numeros[i])`	scanf recebe o endereço de numeros[i] e grava nele o valor digitado.
18	43	**Indireta**	`ordena(numeros, TAM)`	O endereço do vetor é copiado para v e o valor de TAM para n.
19	44	**Direta**	`resultado = …`	resultado recebe o valor retornado por soma.
20	44	**Indireta**	`soma(numeros, TAM)`	O endereço do vetor vai para v e o valor de TAM para n; o return (linha 30) traz o valor de volta.
21	45	**Direta**	`media = (float) resultado / TAM`	media recebe a divisão em ponto flutuante.
22	47	**Indireta**	`printf("Vetor ordenado:")`	O endereço do texto é copiado para o parâmetro de printf.
23	48	**Direta**	`i = 0`	i recebe 0.
24	48	**Direta**	`i = i + 1`	i recebe o próprio valor mais 1.

#	Linha	Tipo	Trecho	Explicação
25	49	Indireta	`printf(" %d", numeros[i])`	O formato e o valor de numeros[i] são copiados para os parâmetros de printf.
26	51	Indireta	`printf("\nSoma…", resultado, media)`	O formato, resultado e media são copiados para os parâmetros de printf.
27	53	Indireta	`return 0`	main devolve 0 ao sistema operacional (execução sem erros).

Tabela 1 – Atribuições identificadas, agrupadas por função.

6. Resultados

O quadro a seguir consolida a contagem por função. As atribuições indiretas de uma chamada são contadas na função que *chama* (por exemplo, a chamada a `troca` pertence a `ordena`), e o `return` pertence à função que o contém.

Função	Diretas	Indiretas	Total
troca	3	0	3
ordena	4	1	5
soma	4	1	5
main	6	8	14
Total do programa	**17**	**10**	**27**

Tabela 2 – Quantidade de atribuições por função.

7. Conclusão

O programa `ordena_vetor.c` possui **27 atribuições**: **17 diretas**, realizadas com o operador `=` (inicializações, contadores dos laços, acumulação da soma, troca de valores e cálculo da média), e **10 indiretas**, decorrentes de passagem de parâmetros em chamadas de função e de retornos (`return total` e `return 0`). Observa-se que a função `main` concentra a maior parte das indiretas por realizar as chamadas a `printf`, `scanf`, `ordena` e `soma`, enquanto `troca` e `ordena`, por serem `void`, não possuem `return`.

Verificação. Diretas: 3 + 4 + 4 + 6 = 17. Indiretas: 0 + 1 + 1 + 8 = 10. Total: 17 + 10 = 27.